



**CENTRO DE PESQUISA, DESENVOLVIMENTO TECNOLÓGICO E INOVAÇÃO
EMBARCATECH**

THEÓGENES GABRIEL ARAÚJO DE ANDRADE

RELATÓRIO FINAL DE MONITORIA: PROJETOS DESENVOLVIDOS

JUAZEIRO - BA

2026

RELATÓRIO FINAL DE MONITORIA: PROJETOS DESENVOLVIDOS

Relatório apresentado como parte das atividades da residência em FPGAs do programa EmbarcaTech, documentando o desenvolvimento dos Projetos Desenvolvidos ao longo das mentorias. Realizei este trabalho sob a orientação do mentor Reinaldo Götz de Oliveira Junior, com o objetivo de aplicar conceitos avançados de lógica programável e sistemas digitais.

RESUMO

Este relatório descreve o conjunto de projetos desenvolvidos durante o período de monitoria na disciplina de Sistemas Digitais. Os projetos abordam a implementação de sistemas embarcados em FPGAs (Field-Programmable Gate Arrays), com foco em processamento de vídeo, comunicação com sensores e implementação de protocolos de comunicação serial. As implementações foram realizadas predominantemente em linguagem de descrição de hardware Verilog, com suporte de código em C para sistemas com processador soft-core Microblaze. O repositório de código documenta o desenvolvimento de interfaces de vídeo HDMI, a integração com o sensor inercial MPU6050, a captura de imagens com a câmera OV2640 e a implementação dos protocolos I2C, SPI e UART. O trabalho demonstra a aplicação prática dos conceitos teóricos da disciplina na solução de problemas de engenharia de computação.

RESUMO.....	3
1. INTRODUÇÃO.....	5
2. PROJETOS DE INTERFACE DE VÍDEO HDMI.....	6
2.1 Gerador de Barras de Cores.....	6
2.1.1 Fundamentação Teórica.....	7
2.1.2 Metodologia de Implementação:.....	7
2.1.3 Módulos utilizados no Sistema.....	7
2.2 Detecção de Bordas em Imagem.....	8
2.2.1 Fundamentação Teórica:.....	9
2.2.2 Metodologia de Implementação.....	10
2.2.2 Módulos utilizados no Sistema.....	11
3. PROJETO COM PROCESSADOR MICROBLAZE.....	11
3.1 Interface com Sensor Inercial MPU6050.....	12
3.1.1 Fundamentação Teórica.....	12
3.1.2 Resumo do funcionamento do Sistema.....	13
4. PROJETOS COM CÂMERA OV2640.....	13
4.1 Captura e Exibição de Vídeo.....	14
4.1.1. Fundamentação Teórica.....	14
4.1.2. Módulos utilizados no Sistema.....	15
4.2 Captura de Vídeo com Filtro de Imagem.....	16
4.2.1. Arquitetura e Componentes:.....	16
5. IMPLEMENTAÇÃO DE PROTOCOLOS DE COMUNICAÇÃO.....	17
5.1 Protocolo I2C (Inter-Integrated Circuit).....	17
5.1.1. Conversor Analógico-Digital ADS1115.....	18
5.1.2. Metodologia de Implementação.....	18
5.1.3 Módulos utilizados no Sistema.....	19
5.2.1. Conversor Analógico-Digital MPU6050.....	20
5.2.2. Metodologia de Implementação.....	20
5.2.3 Módulos utilizados no Sistema.....	21
5.2 Protocolo SPI (Serial Peripheral Interface).....	22
5.2.1. Conversor BMP280.....	22
5.2.2. Metodologia de Implementação.....	23
5.2.2. Módulos utilizados no Sistema.....	24
5.3 Protocolo UART (Universal Asynchronous Receiver-Transmitter).....	24
5.3.1. Fundamentação Teórica.....	25
5.3.2. Módulos utilizados no sistema.....	25
6. CONCLUSÃO.....	26

1. INTRODUÇÃO

O presente relatório tem como objetivo apresentar de forma descritiva os projetos desenvolvidos no âmbito da monitoria da disciplina de Sistemas Digitais. A monitoria visou aprofundar os conhecimentos em projetos de hardware digital, utilizando FPGAs como plataforma de desenvolvimento e a linguagem Verilog para descrição de hardware.

Os projetos foram concebidos para explorar diferentes facetas dos sistemas embarcados, desde a geração e processamento de sinais de vídeo até a comunicação com periféricos e sensores externos. A complexidade dos projetos é crescente, partindo de implementações puramente em hardware até sistemas que integram processadores soft-core para tarefas de controle e processamento de dados mais complexas.

O repositório de código-fonte, disponível em, serve como base para a análise aqui apresentada e contém todos os módulos Verilog, códigos em C, arquivos de constraints e documentação de suporte desenvolvidos.

Este documento está estruturado em capítulos que correspondem às principais categorias de projetos desenvolvidos: interfaces de vídeo HDMI, sistemas com processador Microblaze, projetos com câmera de vídeo e implementações de protocolos de comunicação. Cada capítulo detalha a arquitetura, a funcionalidade e os principais componentes de cada projeto.

2. PROJETOS DE INTERFACE DE VÍDEO HDMI

Os projetos de interface de vídeo HDMI (High-Definition Multimedia Interface) exploram a geração de sinais de vídeo digital em conformidade com o padrão TMDS (Transition Minimized Differential Signaling). Foram desenvolvidos dois projetos principais nesta categoria, ambos utilizando a resolução de 480p como base.

2.1 Gerador de Barras de Cores

A validação de interfaces de vídeo digital exige a geração de sinais que não apenas sigam padrões rigorosos de temporização, mas que também garantam a integridade elétrica durante a transmissão em alta velocidade. O projeto do Gerador de Barras de Cores implementa um pipeline completo de processamento de imagem, transformando coordenadas espaciais em fluxos de dados seriais diferenciais compatíveis com o padrão HDMI.

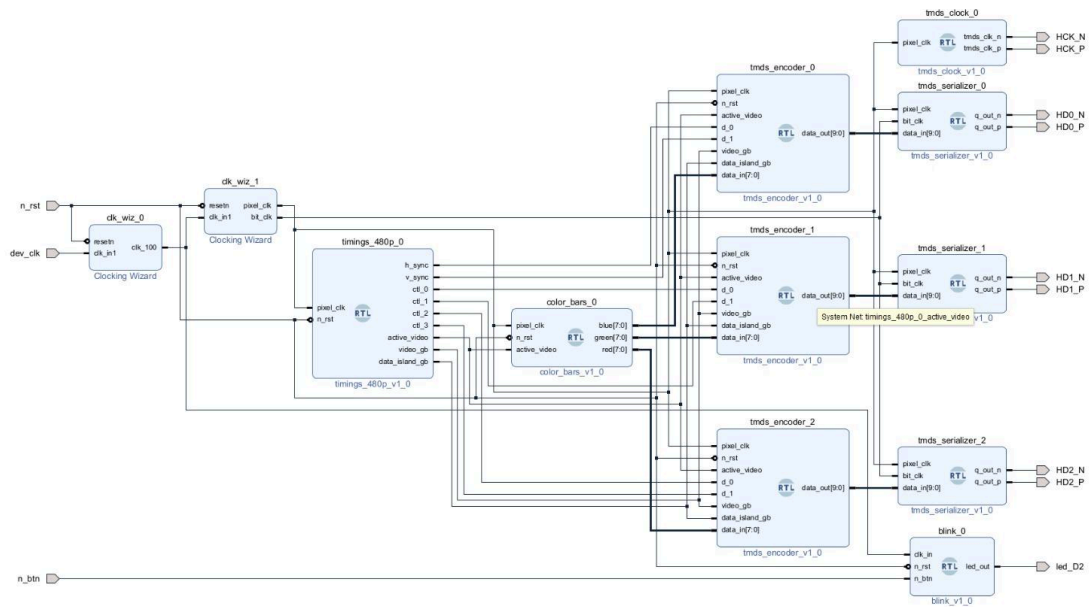


Figura 1. Diagrama funcional do projeto.

Fonte: arquivo pessoal Vivado (2026).

2.1.1 Fundamentação Teórica

O projeto baseia-se na conversão de sinais paralelos RGB de 8 bits para o formato TMDS (Transition Minimized Differential Signaling). Esta tecnologia é fundamental para reduzir a interferência eletromagnética (EMI) e permitir que cabos de cobre transmitam gigabits de informação com baixo erro de bit. A implementação foca na resolução 480p (VGA), que, apesar de ser um padrão legado, é o ponto de partida ideal para a validação de hardware devido à sua estabilidade e ampla compatibilidade com monitores modernos.

2.1.2 Metodologia de Implementação:

Domínio Espacial e Temporal: Onde a tela é mapeada através de contadores horizontais e verticais. A lógica de "barras" é injetada neste estágio, associando a posição do feixe de varredura a um valor cromático específico.

Domínio de Codificação: Onde os dados puramente digitais sofrem uma expansão de 8 para 10 bits. Este processo de codificação não serve apenas para transporte, mas para garantir o "DC-balancing", evitando o acúmulo de carga nos capacitores de acoplamento da interface física.

Domínio de Serialização de Alta Velocidade: O estágio crítico onde o paralelismo do código Verilog encontra os limites físicos da FPGA. Aqui, os dados são transmitidos bit a bit em uma frequência dez vezes superior ao clock de pixel (250 MHz para a resolução 480p), exigindo um controle rigoroso de fase e jitter.

O sistema é composto pelos seguintes módulos Verilog:

2.1.3 Módulos utilizados no Sistema

- **colorBars.v:** Módulo principal que gera as barras de cores (branca, amarela, ciano, verde, magenta, vermelha e azul) com base nos sinais de temporização de vídeo.
- **timings_480p.v:** Responsável pela geração dos sinais de sincronismo horizontal (h_sync) e vertical (v_sync), bem como do sinal de vídeo ativo (active_video), para a resolução de 640x480 pixels a 60Hz.
- **tmds_encoder.v:** Implementa o algoritmo de codificação TMDS, que minimiza as transições de sinal e balanceia a corrente contínua nos pares diferenciais do cabo HDMI.
- **tmds_serializer.v:** Serializa os dados de 10 bits codificados pelo TMDS para transmissão em alta velocidade.
- **tmds_clock.v:** Gera o clock TMDS, que é 5 vezes a frequência do clock de pixel para a serialização 10:1. Funcionamento: O módulo timings_480p.v gera os sinais de temporização que definem a varredura da tela. O módulo colorBars.v utiliza esses sinais para determinar a posição do pixel na tela e, com base nisso, seleciona a cor da barra correspondente. Os dados de cor (RGB) são então codificados pelo tmds_encoder.v, serializados pelo tmds_serializer.v e transmitidos pela interface física da FPGA.

2.2 Detecção de Bordas em Imagem

Este projeto avança em relação ao anterior, introduzindo o processamento de imagem em tempo real. O sistema é capaz de receber uma imagem via comunicação serial UART, armazená-la em um buffer de frames e aplicar um filtro de detecção de bordas (Sobel) antes de exibi-la na saída HDMI.

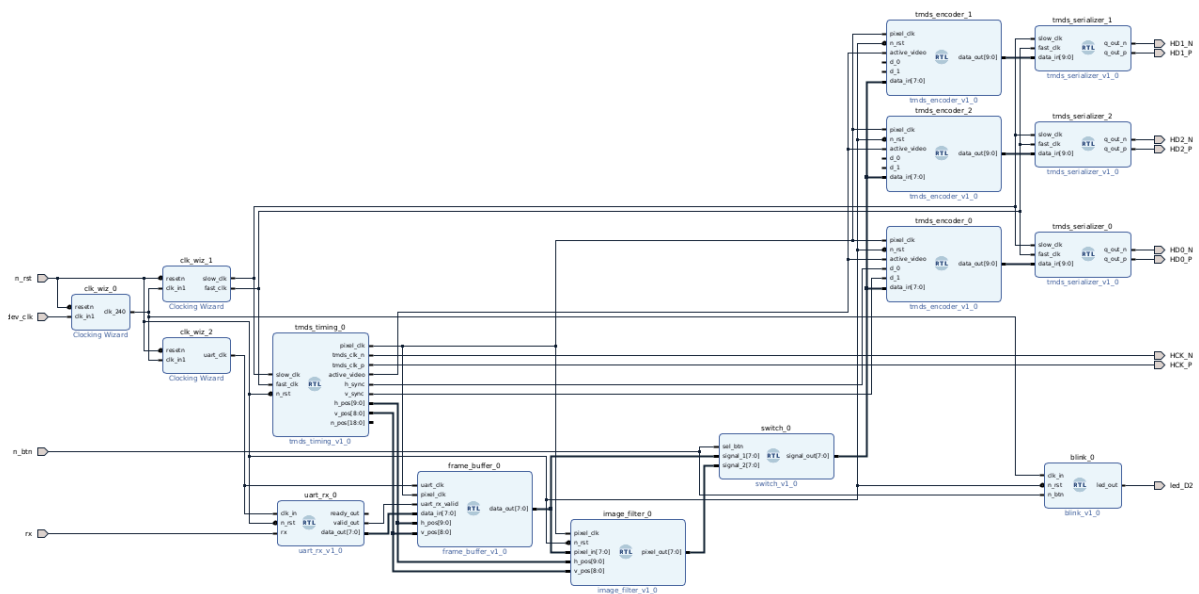


Figura 2. Diagrama funcional do projeto.
Fonte: arquivo pessoal Vivado (2026).

2.2.1 Fundamentação Teórica:

A UART é um protocolo de comunicação serial assíncrona. Diferente do HDMI, ela não possui um clock compartilhado, exigindo que o receptor utilize uma técnica de **oversampling** (geralmente 16x a taxa de baud rate) para amostrar o dado no centro de cada bit, garantindo a integridade da recepção mesmo com pequenas variações de frequência entre os dispositivos.

Devido à diferença de velocidade entre a chegada dos dados (UART) e a exibição (HDMI), é indispensável o uso de um *Frame Buffer*. Em FPGAs, isso é implementado através de **Block RAMs (BRAM)**. O buffer atua como uma memória de porta dupla (Dual-Port), onde um processo escreve os pixels recebidos e outro lê para a exibição, permitindo o isolamento de domínios de clock (*Clock Domain Crossing*).

O operador Sobel é um filtro linear de diferenciação, utilizado para calcular uma aproximação do gradiente da intensidade de uma imagem.

- **Convolução:** O processo consiste em deslizar uma máscara (kernel) 3x3 sobre a imagem.
- **Gradientes (Gx e Gy):** A máscara calcula a variação de brilho nas direções horizontal e vertical.
- **Magnitude:** A borda é identificada onde a magnitude do gradiente é alta. No $|G| = \sqrt{G_x^2 + G_y^2}$ projeto, utiliza-se a simplificação $|G| \approx |G_x| + |G_y|$ para economizar recursos de hardware (evitando raízes quadradas e potências).

2.2.2 Metodologia de Implementação

A implementação do filtro Sobel em hardware requer o acesso simultâneo a pixels de diferentes linhas da imagem. Como a leitura ocorre de forma serial, são utilizados line buffers para armazenar as duas linhas anteriores, permitindo que, a cada ciclo de clock, o sistema forme uma janela 3x3 com a linha atual e as duas anteriores. Esse mecanismo viabiliza a aplicação eficiente do filtro em tempo real, garantindo o acesso contínuo à vizinhança necessária para o cálculo do gradiente.

O processamento é organizado em um pipeline de três estágios: no primeiro, ocorre o deslocamento dos pixels na janela 3x3; no segundo, são aplicadas as máscaras de Sobel para calcular as componentes (Dx) e (Dy); e no terceiro, é calculada a magnitude do gradiente, gerando a imagem de bordas. A integração do sistema é feita pelos módulos switch.v e top, permitindo a seleção entre o modo bypass e o modo filtrado via botão físico. O sinal final é enviado ao tmds_encoder.v para transmissão HDMI, com o mapeamento de pinos definido no arquivo colorlight.xdc, incluindo clock de 25 MHz, UART e pares diferenciais TMDS.

2.2.2 Módulos utilizados no Sistema

Além dos módulos TMDS e de temporização já mencionados, este projeto inclui:

- **image_filter.v:** Implementa o filtro de detecção de bordas, que calcula o gradiente da imagem para realçar as transições de intensidade luminosa.
- **frame_buffer.v:** Um módulo de memória RAM de dupla porta (Dual-Port RAM) que armazena a imagem recebida, permitindo que a escrita (via UART) e a leitura (para exibição HDMI) ocorram de forma independente.
- **uart_rx.v:** Módulo receptor UART que decodifica os dados seriais recebidos e os disponibiliza para serem escritos no buffer de frames. Funcionamento: Uma imagem é enviada para a FPGA através da interface UART. O módulo `uart_rx.v` recebe os pixels da imagem e os armazena sequencialmente no `frame_buffer.v`. Concomitantemente, o sistema de vídeo lê os dados do buffer, aplica o filtro de detecção de bordas através do módulo `image_filter.v` e envia a imagem processada para a saída HDMI. O resultado é a exibição da imagem original com suas bordas destacadas.

3. PROJETO COM PROCESSADOR MICROBLAZE

Para explorar a integração entre hardware e software em sistemas embarcados, foi desenvolvido um projeto utilizando o processador soft-core Microblaze da Xilinx. Este processador é sintetizado no próprio tecido da FPGA, permitindo a execução de código em C e a interação com periféricos de hardware customizados.

3.1 Interface com Sensor Inercial MPU6050

3.1.1 Fundamentação Teórica

O projeto apresenta uma implementação consistente de um sistema de aquisição de dados inerciais, integrando o processador MicroBlaze — um soft-core da Xilinx — ao sensor MPU6050 por meio do protocolo I²C, utilizando a biblioteca padrão `xiic.h`. A arquitetura adotada faz uso de periféricos consolidados da Xilinx, como o `XUartLite`, empregado para depuração e visualização dos dados de telemetria, o `Xlic`, responsável pela comunicação mestre-escravo com o sensor inercial, e o `XGpio`, possivelmente utilizado para controle de LEDs ou leitura de botões, conforme indicado no arquivo de restrições de pinos.

Inicialmente, o registrador `PWR_MGMT_1` é configurado para retirar o dispositivo do modo de baixo consumo (`sleep`). Em seguida, a leitura sequencial dos dados é realizada a partir do registrador `ACCEL_XOUT_H` (`0x3B`), estratégia eficiente que explora o incremento automático de endereços do MPU6050, garantindo a coerência temporal entre as amostras de aceleração e giroscópio. No entanto, observa-se que o cálculo das grandezas físicas assume a configuração padrão do acelerômetro em $\pm 2g$, utilizando o fator de escala de 16384 LSB/g. Caso o registrador `ACCEL_CONFIG` seja alterado, torna-se necessário ajustar o divisor correspondente para manter a correta conversão dos valores.

Por fim, o arquivo de restrições `colorlight.xdc` define a pinagem para uma placa da família Colorlight, amplamente utilizada em projetos com FPGA. O clock de sistema é fornecido por um sinal de 25 MHz, enquanto a comunicação UART utiliza os pinos M3 (RX) e R3 (TX). Entretanto, não há definição explícita dos pinos SDA e SCL referentes à interface I²C. Dessa forma, é fundamental garantir que os pinos atribuídos no Vivado ao bloco IIC estejam corretamente conectados fisicamente ao MPU6050, preferencialmente por meio dos conectores disponíveis na placa, como os headers P2, P5 ou P6, assegurando a confiabilidade da comunicação entre o processador e o sensor.

3.1.2 Resumo do funcionamento do Sistema

- **Processador Microblaze:** O núcleo do sistema, responsável por executar o software de controle.
- **Periférico I2C:** Um módulo de hardware (IP core) que implementa o protocolo de comunicação I2C, utilizado para a comunicação com o MPU6050.
- **Periférico UART:** Utilizado para a comunicação com um computador host, permitindo a visualização dos dados do sensor em um terminal serial.
- **Código em C (MPU6050.c):** O software é executado pelo Microblaze. Este código é responsável por: Inicializar o periférico I2C, Configurar os registradores internos do MPU6050 para definir a sensibilidade do acelerômetro e do giroscópio, Realizar leituras periódicas dos registradores de dados do sensor, Processar os dados brutos lidos para convertê-los em valores de aceleração (em g) e velocidade angular (em graus/s) e Enviar os dados processados para o computador host via UART. Após a inicialização, o programa em C entra em um laço infinito. A cada iteração, o Microblaze comanda o periférico I2C para ler os valores dos eixos X, Y e Z do acelerômetro e do giroscópio do MPU6050. Os dados lidos são então processados e formatados para serem enviados pela UART. Um programa de terminal no computador host pode então receber e exibir esses dados, permitindo a monitoração em tempo real do movimento e da orientação do sensor.

4. PROJETOS COM CÂMERA OV2640

Os projetos com a câmera OV2640 focam na captura e processamento de vídeo em tempo real. A OV2640 é um sensor de imagem CMOS de 2 megapixels que pode ser configurado para diversos formatos de saída de vídeo. A comunicação e configuração da câmera são realizadas através do protocolo SCCB (Serial Camera Control Bus), uma variante do I2C.

4.1 Captura e Exibição de Vídeo

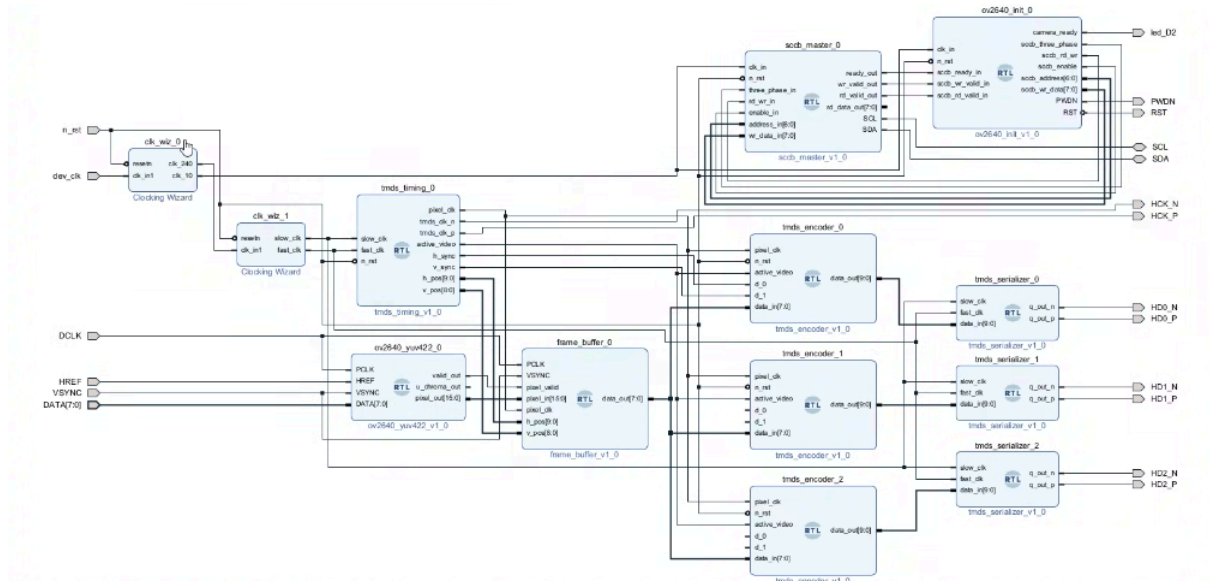


Figura 3. Diagrama funcional do projeto.

Fonte: arquivo pessoal Vivado (2026).

O projeto inicial com a câmera OV2640 implementa um sistema capaz de capturar o fluxo de vídeo da câmera e exibi-lo em tempo real em um monitor através da interface HDMI. Arquitetura e Componentes:

4.1.1. Fundamentação Teórica

O **SCCB (Serial Camera Control Bus)** é um protocolo proprietário da OmniVision, muito similar ao I2C, utilizado para configurar os registradores internos do sensor OV2640. Ele permite definir parâmetros como formato de saída (YUV/RGB), balanço de branco e exposição. No código `sccb_master.v`, a lógica gerencia as fases de escrita e leitura de 3 fios para inicializar a câmera.

Ao contrário do RGB, o formato **YUV** separa a luminância (Y) da crominância (U e V). O padrão **4:2:2** é uma técnica de subamostragem onde a luminância é transmitida para cada pixel, mas a informação de cor é compartilhada entre dois

pixels adjacentes. Isso reduz a largura de banda necessária sem perda perceptível de qualidade para o olho humano. O módulo `ov2640_yuv422.v` é o responsável por reconstruir esses componentes a partir do barramento de 8 bits da câmera.

Para a transmissão via HDMI, utiliza-se o **TMDs**. Esta tecnologia utiliza algoritmos de 8b/10b para minimizar as transições de estado nos fios de cobre, reduzindo a interferência eletromagnética (EMI) e garantindo o balanço DC da linha.

O módulo `tmds_encoder.v` realiza essa transformação, enquanto o `tmds_serializer.v` converte os dados paralelos em um fluxo serial de alta velocidade para os pares diferenciais.

4.1.2. Módulos utilizados no Sistema

- **ov2640_init.v**: Módulo responsável por inicializar e configurar a câmera OV2640 através do protocolo SCCB. Ele escreve em uma série de registradores da câmera para definir a resolução, o formato de cor (YUV422) e outras configurações de imagem.
- **sccb_master.v**: Implementação em Verilog do mestre do protocolo SCCB, utilizada pelo `ov2640_init.v` para se comunicar com a câmera.
- **ov2640_yuv422.v**: Módulo que recebe os dados de vídeo no formato YUV422 da câmera e os converte para o formato RGB, que é o formato de cor padrão para a maioria dos monitores.
- **frame_buffer.v**: Utilizado para sincronizar o fluxo de vídeo da câmera com o tempo da saída HDMI.

Este projeto é uma extensão do anterior, adicionando a capacidade de aplicar filtros de processamento de imagem em tempo real ao vídeo capturado. A arquitetura é similar à do projeto de detecção de bordas em imagem estática, mas adaptada para operar em um fluxo de vídeo contínuo.

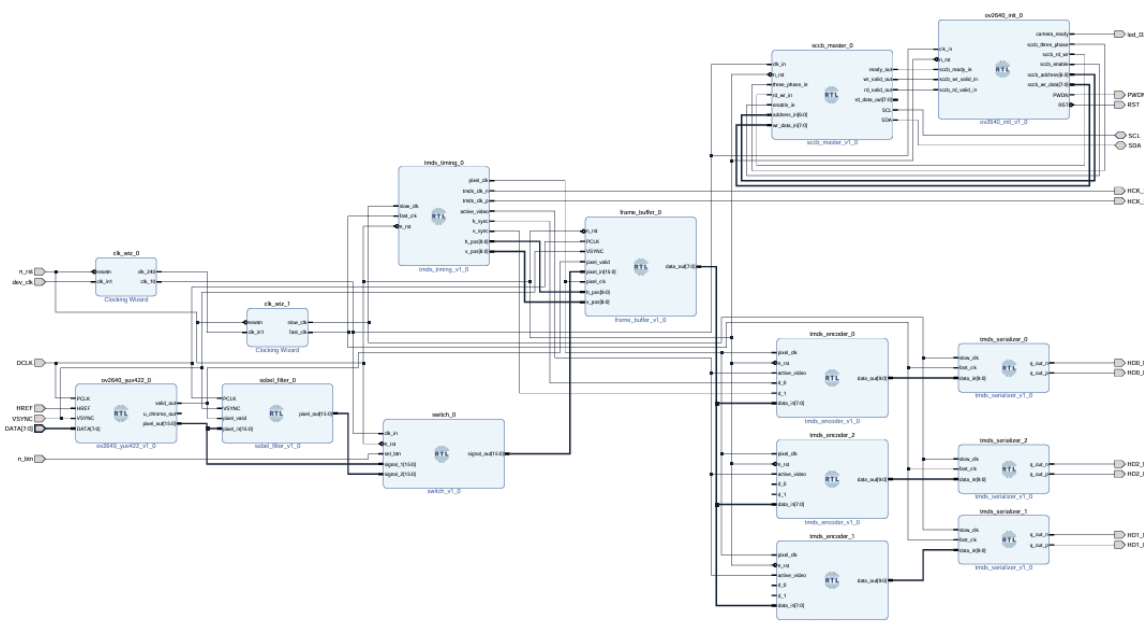


Figura 4. Diagrama funcional do projeto.
Fonte: arquivo pessoal Vivado (2026).

4.2.1. Arquitetura e Componentes:

O sistema reutiliza a maioria dos componentes do projeto anterior, com a adição de um módulo de filtro de imagem, similar ao `image_filter.v`, inserido no caminho de dados entre o conversor YUV-RGB e o buffer de frames. Isso permite que o filtro seja aplicado a cada quadro de vídeo antes de ser exibido. O fluxo de vídeo da câmera é capturado e convertido para RGB. Antes de ser escrito no buffer de frames, cada quadro passa pelo módulo de filtro de imagem. O vídeo filtrado é então armazenado e exibido na saída HDMI. Isso permite a aplicação de efeitos como detecção de bordas, suavização ou outros filtros de convolução diretamente no vídeo ao vivo.

5. IMPLEMENTAÇÃO DE PROTOCOLOS DE COMUNICAÇÃO

Uma parte fundamental do desenvolvimento de sistemas embarcados é a comunicação com o mundo exterior. Diversos projetos desenvolvidos nesta monitoria envolveram a implementação e o uso de protocolos de comunicação serial padrão da indústria.

5.1 Protocolo I2C (Inter-Integrated Circuit)

O protocolo I2C é um barramento serial de dois fios (SDA para dados e SCL para clock) amplamente utilizado para a comunicação entre microcontroladores e periféricos de baixa velocidade em curtas distâncias. No contexto dos projetos desenvolvidos, o I2C foi crucial para a comunicação com o sensor inercial MPU6050. Foi implementado um mestre I2C em software (executado no Microblaze) para enviar comandos e ler dados do sensor.

5.1.1. Conversor Analógico-Digital ADS1115

O projeto utiliza o conversor analógico-digital ADS1115, um dispositivo de 16 bits de alta precisão. A teoria de operação deste componente envolve a manipulação de registradores internos, especificamente o registrador de configuração, que define parâmetros como o ganho do amplificador programável (PGA), a taxa de amostragem e a seleção de canais de entrada. Complementarmente, utiliza-se a norma UART (Universal Asynchronous Receiver-Transmitter) para a exportação dos dados. Diferente do I2C, a UART é assíncrona e exige a pré-configuração de uma taxa de transmissão (Baud Rate) comum entre os dispositivos para garantir a correta amostragem dos bits de dados, start e stop.



Figura 5. Módulos ADS1115.
Fonte: arquivo da internet (2026).

5.1.2. Metodologia de Implementação

A metodologia adotada consistiu no desenvolvimento de um sistema digital modular em linguagem de descrição de hardware **Verilog**. O processo iniciou-se pela implementação de controladores de baixo nível para as interfaces físicas, garantindo que as temporizações do I2C e da UART respeitassem as frequências de operação definidas para o sistema. Para a gestão do fluxo de dados, foi

projetada uma Máquina de Estados Finitos (FSM) centralizada, responsável por orquestrar a sequência lógica de inicialização do sensor ADS1115, a captura cíclica das conversões e o subsequente reencaminhamento desses valores para o módulo de transmissão serial.

A integração do hardware foi realizada através de um arquivo de restrições (**XDC**), mapeando os sinais lógicos do código para os pinos físicos da placa de desenvolvimento (Colorlight). Este mapeamento incluiu a definição do clock de entrada de 25 MHz e a atribuição dos pinos específicos para as linhas de dados do sensor e da interface de depuração serial. Para assegurar a estabilidade do sistema no momento da ativação, foi implementado um módulo de *Power-on Reset*, que mantém o sistema em estado de repouso durante os primeiros ciclos de clock, evitando comportamentos erráticos durante a estabilização da tensão de alimentação.

5.1.3 Módulos utilizados no Sistema

O projeto é composto por cinco módulos principais que interagem entre si:

- **i2c_master.v**: Gerencia o protocolo físico I2C, incluindo geração de *Start/Stop*, arbitragem de clock e tratamento de *Acknowledge* (ACK/NACK).
- **ADS1115_ctrl.v**: O "cérebro" do projeto. Coordena quando escrever no ADC para configurá-lo e quando ler os dados, servindo de ponte entre o I2C e a UART.
- **uart_tx.v**: Recebe os dados de 8 bits processados e os serializa para transmissão via protocolo UART.
- **uart_rx.v**: Responsável por receber comandos externos (se aplicável) via interface serial.
- **power_on_rst.v**: Garante um pulso de reset estável de 8 ciclos de clock durante a inicialização do chip.
- **colorlight.xdc**: Arquivo de restrições que mapeia os sinais para os pinos físicos (ex: SDA no pino T3, SCL no pino U2).

5.2.1. Conversor Analógico-Digital MPU6050

A base teórica deste projeto reside na integração de sistemas digitais para aquisição de dados inerciais e comunicação serial. O sistema utiliza o protocolo I2C (Inter-Integrated Circuit), uma interface de comunicação síncrona de dois fios (SDA para dados e SCL para clock), que permite a comunicação mestre-escravo entre a FPGA e o sensor MPU6050. Este protocolo é fundamental por permitir o endereçamento de múltiplos dispositivos e o controle de fluxo de dados de forma robusta, essencial para configurar registradores internos e extrair amostras de sensores complexos. Além disso, utiliza-se a UART (Universal Asynchronous Receiver-Transmitter), um protocolo assíncrono que possibilita a transmissão de dados para um computador ou terminal externo. A UART opera sem um clock compartilhado, baseando-se em taxas de transmissão (baud rates) e bits de sincronização (start e stop) para garantir que os bytes de informação sejam interpretados corretamente pelo receptor.

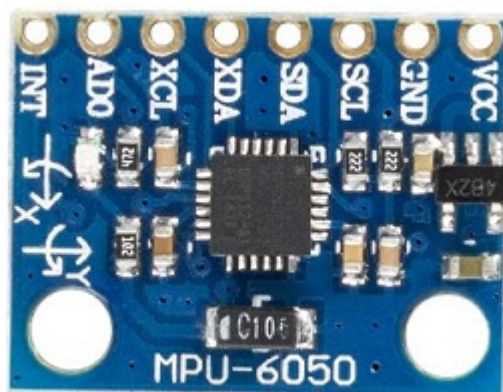


Figura 6. Módulos MPU-6050.
Fonte: arquivo da internet (2026).

5.2.2. Metodologia de Implementação

A metodologia adotada para a implementação do sistema na FPGA baseou-se no design modular e no uso intensivo de Máquinas de Estados Finitos (FSM). Cada protocolo de comunicação foi isolado em seu próprio módulo, permitindo testes independentes antes da integração final. Para garantir a estabilidade do hardware, foi implementada uma lógica de Power-on Reset, que mantém o sistema em estado seguro durante os primeiros ciclos de clock após a energização. A temporização dos sinais foi calculada através de parâmetros de oversampling, permitindo que o sistema se adapte a diferentes frequências de clock principal (como os 10 MHz definidos no código) e mantenha a integridade dos sinais I2C e UART. O mapeamento físico foi realizado através de um arquivo de restrições XDC, definindo os padrões elétricos LVCMOS33 para os pinos da placa Colorlight, garantindo a compatibilidade de tensão entre a FPGA e o sensor periférico.

5.2.3 Módulos utilizados no Sistema

O projeto é composto por cinco módulos principais que interagem entre si:

- **i2c_master:** Atua como o controlador do barramento I2C. Ele gerencia as condições de START, STOP e gera o sinal de clock (SCL) com base na frequência de entrada, além de lidar com os sinais de reconhecimento (ACK/NACK).
- **MPU6050_ctrl:** É o cérebro do sistema. Este módulo coordena as requisições ao i2c_master, definindo quais endereços de registradores ler, armazena temporariamente os bytes recebidos em um buffer e os encaminha para o transmissor UART.
- **uart_tx:** Módulo responsável pela transmissão serial. Ele recebe um byte do controlador do MPU6050 e o serializa, adicionando os bits de start e stop para envio via pino TX.
- **uart_rx:** Responsável por receber comandos ou dados seriais externos. Ele utiliza uma técnica de oversampling (16x por padrão) para amostrar o pino RX e reconstruir o byte de dado original.

- **power_on_rst:** Garante a integridade da inicialização do sistema, gerando um sinal de reset ativo baixo (`n_rst_out`) durante os primeiros instantes de operação do hardware.

5.2 Protocolo SPI (Serial Peripheral Interface)

O SPI é um protocolo de comunicação serial síncrono que opera em modo full-duplex. Ele utiliza quatro fios: MISO (Master In, Slave Out), MOSI (Master Out, Slave In), SCLK (Serial Clock) e SS (Slave Select). Embora não tenha sido o protocolo principal em um dos projetos finalizados, sua implementação foi estudada e desenvolvida como um módulo Verilog genérico, pronto para ser integrado em futuros projetos que necessitem de comunicação de alta velocidade com periféricos como memórias flash, conversores A/D ou outros microcontroladores.

5.2.1. Conversor BMP280

A base teórica deste projeto fundamenta-se na comunicação entre sistemas digitais e sensores MEMS (*Micro-Electro-Mechanical Systems*). O protocolo central utilizado é o **SPI (Serial Peripheral Interface)**, uma interface síncrona que opera no modelo mestre-escravo através de quatro sinais principais: SCLK, MOSI, MISO e SS. Este protocolo é escolhido pela sua alta velocidade de transferência e simplicidade de implementação em hardware, permitindo o acesso direto aos registradores internos de pressão e temperatura do sensor BMP280. Complementarmente, utiliza-se o protocolo **UART (Universal Asynchronous Receiver-Transmitter)** para a ponte de dados entre a FPGA e o computador, permitindo a visualização dos resultados em um terminal serial.

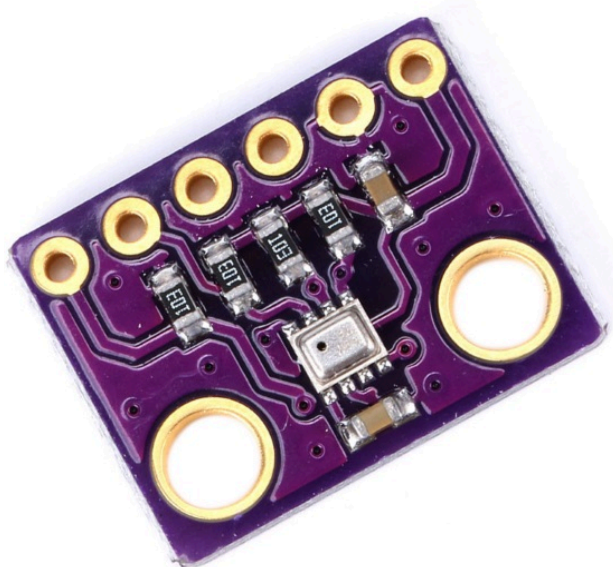


Figura 7. Módulos BMP280.
Fonte: arquivo internet (2026).

5.2.2. Metodologia de Implementação

A metodologia adotada para o desenvolvimento do sistema seguiu um fluxo de projeto digital padrão para FPGAs, focado na modularização e verificabilidade. Inicialmente, o hardware foi segmentado em blocos funcionais, onde cada componente (SPI, UART, Divisores de Clock) foi descrito em linguagem Verilog e SystemVerilog. O desenvolvimento priorizou a criação de um controlador centralizado, o `bmp280_ctrl.v`, que atua como o "cérebro" da aplicação, gerenciando o fluxo de informações entre o sensor físico e a interface de saída para o usuário.

Para garantir a confiabilidade do sistema antes da implementação física, foram realizados testes funcionais através de testbenches em ambiente de simulação. Nestas simulações, o comportamento do sensor BMP280 foi emulado para verificar se a máquina de estados respondia corretamente aos sinais de pronto e válido. Após a validação lógica, o projeto passou pelo processo de síntese e

mapeamento de pinagem utilizando o arquivo de restrições físicas (.cst), sendo então carregado na placa Tang Nano 9K. A verificação final foi feita através da análise de sinais em tempo real e monitoramento do terminal serial para validar a precisão dos dados recebidos.

5.2.2. Módulos utilizados no Sistema

- **top.v:** Unidade de nível superior que interconecta todos os submódulos e gerencia a pinagem externa.
- **pll.v:** Configura o recurso **PLLVR** para elevar o clock de entrada de 27 MHz para 120 MHz.
- **clock_div.v:** Divisor de frequência para gerar o clock de *oversampling* necessário para a UART (16x o Baud Rate).
- **spi_master.v:** Implementação do protocolo SPI, permitindo o envio e recebimento de múltiplos bytes de forma sequencial.
- **bmp280_ctrl.v:** Máquina de estados principal que define a sequência de comandos para o sensor (leitura de ID, configuração e captura de temperatura/pressão).
- **uart_tx.v / uart_rx.v:** Módulos responsáveis pela transmissão e recepção de caracteres para interface com o terminal serial do PC.
- **spi_controller.sv:** Abstração adicional para gerenciar palavras de dados e o sinal de *Slave Select* (SS) de forma automatizada.

5.3 Protocolo UART (Universal Asynchronous Receiver-Transmitter)

O protocolo UART é um dos métodos mais simples e comuns para comunicação serial assíncrona. Ele requer apenas dois fios, um para transmissão (TX) e um para recepção (RX). No projeto de detecção de bordas em imagem, o UART foi utilizado para enviar a imagem do computador para a FPGA. No projeto com o Microblaze, ele serviu como canal de depuração e para o envio dos dados do

sensor MPU6050 de volta para o computador. Módulos Verilog para recepção (uart_rx.v) e transmissão (uart_tx.v) foram desenvolvidos e utilizados.

5.3.1. Fundamentação Teórica

O sistema desenvolvido fundamenta-se no protocolo de comunicação serial UART (Universal Asynchronous Receiver-Transmitter), um padrão amplamente utilizado para a troca de dados entre dispositivos digitais sem a necessidade de uma linha de clock compartilhada. A teoria da UART baseia-se na sincronização por meio de uma taxa de transmissão pré-definida, conhecida como baud rate, onde cada pacote de dados é emoldurado por um start bit (nível lógico baixo) e um ou mais stop bits (nível lógico alto). Para garantir a precisão na recepção, utiliza-se a técnica de oversampling (sobreamostragem), geralmente a 16 vezes a taxa de bits, o que permite ao recetor identificar o centro de cada bit e minimizar erros causados por desvios de frequência ou ruído na linha.

5.3.2. Módulos utilizados no sistema

O sistema é composto por cinco módulos principais integrados num módulo de topo (top):

pll.v: Este módulo é responsável pelo condicionamento de clock, utilizando as primitivas de hardware da FPGA para gerar uma base de tempo de 120 MHz a partir do cristal oscilador de 27 MHz.

clock.v: Atua como um divisor de frequência programável que, baseado num parâmetro de baud rate, gera o sinal de clock necessário para o funcionamento sincronizado da UART.

uart_rx.v: Implementa a lógica de recepção. Ele monitoriza a linha RX à procura de um start bit, utiliza um contador de sobreamostragem para ler os bits de dados no momento ideal e armazena o byte recebido num registrador, sinalizando a validade do dado ao final do processo.

uart_tx.v: Responsável pela transmissão. Este módulo recebe um dado paralelo de 8 bits e, através de uma FSM, serializa-o para a linha TX, inserindo automaticamente os bits de controlo (start/stop) necessários para o protocolo.

top: O módulo de integração que interliga todos os blocos anteriores, mapeando os sinais internos para os pinos físicos de entrada e saída, como o sys_clk, o botão de reset (n_rst), os pinos de comunicação (rx/tx) e os LEDs de diagnóstico da placa.

6. CONCLUSÃO

Os projetos desenvolvidos durante a monitoria de Sistemas Digitais proporcionaram uma valiosa experiência prática na concepção e implementação de sistemas embarcados complexos utilizando FPGAs. A variedade de projetos permitiu explorar desde a manipulação de sinais de vídeo em tempo real com HDMI até a integração de processadores soft-core e a comunicação com sensores e periféricos externos.

A utilização da linguagem Verilog como principal ferramenta de descrição de hardware, complementada pela programação em C para o processador Microblaze, demonstrou a sinergia entre hardware e software na solução de problemas de engenharia. A implementação de protocolos de comunicação padrão como I2C, SPI e UART reforçou a importância da interoperabilidade em sistemas digitais.

O conjunto de projetos documentados neste relatório e disponíveis no repositório de código constitui um portfólio de implementações que não apenas solidifica os conceitos aprendidos em sala de aula da monitoria do professor Reinaldo, mas também serve como base para futuros desenvolvimentos e estudos na área de sistemas embarcados e design de hardware.